

Campus Bites AI — PRD

KENTECH 구성원용 상황 기반 식당·카페·술집 추천 웹앱 Introduction to AI Programming, Assignment 4

이 문서는 실제 저장소 코드(`src/`)와 데이터(`src/data/places.json`)를 기준으로 작성했다. 화면 문구, 추천 점수 규칙, 데이터 개수는 모두 현재 코드 값과 맞춰 두었고, 코드에서 확인할 수 없는 항목은 본문 끝 [확인 필요] 에 따로 모아 두었다.

1. Product Overview

Campus Bites AI는 "오늘 일정 끝나고 밥을 먹을지 카페를 갈지" 같은, KENTECH 구성원이 거의 매일 반복하는 작은 결정을 대신 좁혀 주는 서비스다. 검색창에 키워드를 넣고 리뷰를 뒤지는 방식이 아니라, 지금 상황(예산·이동 거리·분위기·동행)을 다섯 개 항목으로 고르면 그에 맞는 장소 3곳을 바로 띄워 준다.

핵심은 "AI가 골라준다"는 인상을 주되, 실제로는 **외부 생성형 AI(LLM)를 호출하지 않는 설명 가능한 규칙 기반 점수 추천**이라는 점이다. 각 장소가 왜 위로 올라왔는지를 점수 규칙으로 설명할 수 있고, 네트워크 호출 없이 정적 데이터만으로 즉시 동작한다. 추천 영역에도 이 점을 명시해 두었다("외부 생성형 AI가 아닌, 선택 조건과 장소 속성을 비교하는 설명 가능한 규칙 기반 추천입니다").

여기에 구성원들이 직접 별점과 방문 후기를 남기는 후기 섹션을 붙여, 정적 추천을 사람들의 실제 이용 맥락으로 보완한다. 후기는 데모 범위에서 브라우저 `localStorage` 에만 저장된다.

데이터는 나주 혁신도시·빛가람동 생활권의 실제 장소명을 기준으로 모은 **58곳**이다(밥 27, 카페 17, 술 14).

2. Target Users

명시적으로 KENTECH **전체 구성원**을 대상으로 한다. 과거 버전이 "대학생/STUDENTS"로만 좁혀 잡았던 것을 넓혔다.

- 학부생 — 공강에 빠르고 싼 한 끼, 친구와 가벼운 약속
- 대학원생 — 연구실 근처에서 오래 앉아 작업할 카페, 논문 마감 때 혼자 머물 자리
- 교수 — 미팅·손님 응대에 적당한 식당
- 교직원 — 부서 점심, 회식 장소

후기 작성 품의 소속 선택지(`undergraduate / graduate / faculty / staff`)와 소속별 필터가 이 네 그룹을 그대로 반영한다.

3. Project Goals

1. **결정 피로 줄이기.** 검색이 아니라 조건 선택 → 즉시 3곳. 클릭 몇 번으로 끝나야 한다.

2. **설명 가능성.** 추천이 블랙박스가 아니어야 한다. 점수가 어떤 속성 매칭에서 나왔는지 사람이 따라갈 수 있어야 한다.
3. **로컬 현실성.** 가상의 장소가 아니라 실제로 존재하는 빗가람동·나주혁신도시 후보를 쓴다.
4. **구성원 참여.** 추천만 일방적으로 주는 게 아니라, 후기로 데이터를 살피운다.

비목표(이번 범위 밖): 실시간 영업시간/혼잡도 연동, 서버 DB, 로그인, 실제 거리 계산, LLM 호출.

4. Core User Scenario

대학원생 지민이 4시에 세미나가 끝났다. 저녁 약속 전까지 두 시간이 비어서 혼자 앉아 코드를 마저 짜고 싶다.

1. 페이지를 열고 상단 "지금 추천받기"를 누른다.
2. 장소 유형 **카페**, 예산 **1만원 이하**, 거리 **도보 5분**, 분위기 **편하게 오래 있기**, 동행 **혼밥**을 고른다.
3. 오른쪽에 3곳이 뜬다. 카페를 골랐으므로 "공부하기 좋음" 태그가 붙은 곳이 위로 올라온다(이 조합에서 1순위는 헤이키 커피 나주혁신점).
4. 카드의 메뉴·가격대·주소·추천 이유를 보고, 네이버지도/구글맵 링크로 위치를 확인한다.
5. 다녀온 뒤 후기 섹션에서 소속을 "대학원생"으로 두고 별점과 한 줄 후기를 남긴다. 다음에 같은 브라우저로 들어오면 그 후기가 남아 있다.

5. Feature List

우선순위는 Must(없으면 제품이 아님) / Should(있어야 완성도) / Nice(있으면 좋음)로 나눴다.

Must

- **조건 기반 Top-3 추천** — 장소 유형·예산·거리·분위기·동행 5개 셀렉트. 선택 즉시 점수 계산 후 상위 3곳 렌더. (`src/app/page.tsx` 의 `scorePlace` , `results`)
- **유형 단일 보장** — 선택한 장소 유형만 결과에 나온다. 밥을 고르면 카페·술집이 절대 섞이지 않는다. (`results` 가 `placeType === filters.placeType` 로 먼저 필터링)
- **빈 상태 처리** — 유형 필터로 후보가 0개가 되면 "선택한 조건에 맞는 장소가 없어요. 조건을 바꿔보세요." 안내 카드를 띄운다. 1~2개면 빈 슬롯을 억지로 채우지 않고 있는 만큼만 보여 준다. (현재 데이터에선 0개가 나지 않지만 방어적으로 처리)
- **동적 추천 이유** — 카드에 고정 `reason` 문구만 두지 않고, 그 아래 "이 조건이 맞아서 추천: ..." 줄을 추가한다. `scorePlace` 가 점수를 줄 때 어떤 조건이 일치했는지(예산 맞춤, 도보 거리 적합, 혼밥 적합, 공부하기 좋음 등)를 함께 모아 표시한다. → G2(추천 근거 이해)의 직접 근거.
- **실제 장소 데이터** — 58곳, 각 장소에 메뉴/가격/주소/추천 이유/태그. (`src/data/places.json`)
- **지도 링크** — 각 카드에서 네이버지도·구글맵으로 바로 이동. (`createMapLinks`)

Should

- **구성원 후기** — 별점(1~5), 소속, 닉네임, 방문 목적, 본문. `localStorage` 저장, 새로고침해도 유지. (`src/components/reviews-section.tsx`)

- **소속별 후기 필터 / 평균 별점** — 전체·학부생·대학원생·교수·교직원으로 거른다. 상단에 평균 별점과 후기 수 표시.
- **내가 쓴 후기 삭제** — 로컬에 저장된(내가 쓴) 후기에만 삭제 버튼이 뜬다.
- **후기-추천 카드 연결** — 추천 카드마다 그 장소의 후기 수("후기 N개")를 표시하고, "이 장소 후기 쓰기" 버튼을 누르면 후기 섹션으로 스크롤하면서 작성 폼의 장소 select가 그 장소로 자동 선택된다. 후기 상태는 Home 이 들고 있고 (`localReviews`), `ReviewsSection` 은 props로 제어한다. 공유 데이터/타입은 `src/lib/reviews.ts` 로 분리했다.
- **다크 모드** — `next-themes` 하나로 통일. "무드 바꾸기" 버튼이 light/dark를 토글한다.

Nice

- **카페 특화 가중치** — 카페를 고르면 "공부하기 좋음/혼자 가기 좋음" 같은 태그에 추가 점수를 줘서, 학생들이 가장 많이 찾는 "공부 카페"가 먼저 뜨게 한다.
- **샘플 후기 시드** — 후기가 비어 보이지 않도록 3개의 예시 후기를 미리 넣어 둬.
- **반응형 레이아웃** — 모바일·데스크톱 양쪽 대응.

참고: 즐겨찾기/북마크, 로그인, 서버 저장 같은 항목은 이번 범위의 Must/Should가 아니다(미구현이며 비목표).

6. Page Structure

단일 페이지(SPA 느낌의 한 라우트, `/`)에 앵커 네비게이션으로 섹션을 잇는다. 상단 네비: Problem · Features · Recommend · Reviews · Data.

앵커	섹션	내용
<code>#hero</code>	히어로	한 줄 가치 제안, "지금 추천받기" CTA, "무드 바꾸기"(다크 토글), 타깃/입력/추천 요약 카드
<code>#problem</code>	문제 정의	매일 반복되는 장소 선택 피로 + 실제 후보 58곳 스냅샷(밥/카페/술 개수 동적 표시)
<code>#features</code>	핵심 기능	조건 기반 빠른 추천, 다양한 구성원 상황, 구성원 후기 — 3장 카드
<code>#recommend</code>	추천 MVP	왼쪽 5개 셀렉트 + "규칙 기반 추천" 안내, 오른쪽 Top-3 결과 카드(추천 이유 줄 · 지도 링크 · 후기 수 · "후기 쓰기" 버튼). 후보 0개면 안내 카드
<code>#reviews</code>	후기	작성 폼(소속·닉네임·목적·별점·본문) + 소속 필터 + 후기 리스트
<code>#coverage</code>	데이터 현황	총 개수, 밥/카페/술 분포, 현재 단계(MVP) 안내

개수 표기(`totalCount` , `foodCount` , `cafeCount` , `drinkCount`)는 하드코딩이 아니라 `places.json` 을 세서 렌더하므로, 데이터를 늘리면 화면 숫자도 자동으로 따라간다.

7. Technical Requirements

- **프레임워크:** Next.js 16 (App Router, Turbopack), React 19
- **언어:** TypeScript 5 (strict)
- **스타일:** Tailwind CSS 3.4, 다크 모드는 `class` 전략
- **테마:** `next-themes` (`ThemeProvider` attribute="class" defaultTheme="system" enableSystem)
- **아이콘:** lucide-react
- **상태/데이터:** 클라이언트 `useState` 만 사용. 추천은 정적 JSON import. 후기는 `localStorage`. 서버·DB·외부 API 없음.
- **데이터 모델(`src/types/place.ts`):** `Place = { name, placeType, address, budget, distance, mood, group, menu, walk, price, tags[], reason }`. 열거형은 `placeType: food|cafe|drink`, `budget: low|mid|high`, `distance: near|medium|far`, `mood: quick|cozy|trendy`, `group: solo|friend|team`.
- **후기 모델(`src/lib/reviews.ts`):** `Review = { id, placeName, affiliation, nickname, purpose, rating, content, createdAt, isLocal }`, `affiliation: undergraduate|graduate|faculty|staff`, 저장 키 `campus-bites-reviews`, `id`는 `crypto.randomUUID()`. 타입·샘플 데이터·`loadLocalReviews` / `formatDate` 는 이 모듈에 모아 두고 `page.tsx` 와 `reviews-section.tsx` 가 공유한다. 후기 상태(`localReviews`)는 `Home` 이 소유하고 `ReviewsSection` 은 props로 제어된다.
- **스크립트:** `dev = next dev --turbopack`, `build = next build`, `lint = eslint` .
- **배포:** GitHub main에 push → Vercel 자동 배포.

추천 점수 규칙 (코드 그대로)

`scorePlace(place, filters)` 는 매칭마다 점수를 더한다. 가중치는 다음과 같다.

조건	점수
장소 유형 일치	+4
예산 일치	+3
거리 일치	+3
분위기 일치	+2
동행 일치	+2
(보정) 동행=모임인데 장소가 friend	+1
(보정) 거리=가까움인데 장소가 medium	+1
(보정) 예산=1만원대인데 장소가 저예산	+1

여기에 장소 유형으로 "카페"를 골랐을 때만 적용되는 가산점이 따로 있다.

카페 한정 조건	점수
태그 "공부하기 좋음"	+5
태그 "혼자 가기 좋음"	+3
동행이 solo	+2
분위기가 cozy	+2
태그 "대화하기 좋음"	+1
태그 "감각적인 공간"	+1

`scorePlace` 는 점수만 반환하지 않고 `{ score, reasons }` 를 함께 돌려준다. `reasons` 는 점수가 더해질 때마다 그 사유 문구를 모은 배열이고(예산 맞춤 · 도보 거리 적합 · 혼밥 적합 · 공부하기 좋음 ...), 카드의 "이 조건이 맞아서 추천:" 줄이 이 배열을 `.` 로 이어 보여 준다. 데이터(`places.json`)는 건드리지 않고 표시/계산 로직만 추가했다.

최종적으로 `results` 는 (1) 선택한 유형으로 먼저 필터 → (2) 각 장소에 `{ score, reasons }` 부여 → (3) 점수 내림차순 정렬 → (4) 상위 3개를 잘라 보여 준다. 장소 유형 가중치(+4)는 사실상 동률 깨기용으로만 남아 있고, 그래서 유형 일치는 `reasons` 에 넣지 않는다(이미 필터로 보장되니까). 진짜로 유형을 분리하는 건 점수가 아니라 그 앞단의 필터다.

7-A. Acceptance Criteria

추가/개선한 기능이 "됐다"고 말하려면 아래가 만족돼야 한다.

- **유형 단일:** 어떤 조합을 골라도 결과 3곳이 모두 선택한 유형이다. (시뮬레이션 5조합 통과)
- **빈 상태:** 선택 유형의 후보가 0개면 결과 영역에 안내 카드가 뜨고, 카드 그리드는 렌더되지 않는다. 1~2개면 그 수만큼만 보인다.
- **추천 이유:** 각 추천 카드에 `place.reason` (고정) 아래로 "이 조건이 맞아서 추천: ..." 줄이 보이고, 그 문구가 실제로 점수에 기여한 조건과 일치한다. 일치 조건이 하나도 없으면 그 줄은 숨긴다.
- **후기 수:** 추천 카드의 "후기 N개"가 (샘플 + `localStorage`) 후기 중 그 장소명과 일치하는 개수와 같다.
- **후기 연결:** "이 장소 후기 쓰기"를 누르면 `#reviews` 로 스크롤되고, 작성 폼의 장소 `select`가 그 장소로 바뀐다. 같은 장소를 다시 눌러도 동작한다(`nonce`로 재트리거).
- **회귀 없음:** 위를 추가한 뒤에도 `npm run lint` 와 `npm run build` 가 통과한다.

8. Design Requirements

- 따뜻한 크림/오렌지 그라데이션 배경, 둥근 카드와 알약형 버튼, `backdrop blur`로 부드러운 톤.
- 다크 모드에서는 같은 구조를 어두운 남색 계열로 전환. 색은 Tailwind `dark:` 변형으로 처리.
- 강조색은 teal(라벨)과 orange(숫자·별점). 별점은 채워진 별/빈 별로 표현.

- 한글 본문이 주이고 섹션 라벨만 영문 대문자(예: CORE FEATURES). 둘의 톤이 따로 놀지 않게 라벨은 짧게.
- 키보드/스크린리더 고려: 별점 입력은 `role="radiogroup"` , 각 별은 `aria-checked` 와 `aria-label` 을 갖는다. 삭제 버튼도 `aria-label` 부여.
- 모바일에서 1열, 데스크톱에서 다열로 떨어지는 반응형 그리드.

9. Milestones

코드와 커밋에서 확인되는 범위만 적는다. 정확한 일정은 [확인 필요] .

- **M1 — 셋업:** EasyNext 템플릿으로 Next.js 16 프로젝트 생성, 첫 커밋(Initial commit from Create Next App).
- **M2 — 랜딩 + 추천 MVP:** 히어로~데이터 섹션, 셀렉트 5종, `scorePlace` 1차, 초기 장소 데이터. (Build Campus Bites AI landing page)
- **M3 — 데이터 확장:** 빗가람동·나주혁신도시 후보를 58곳까지 늘리고 밥/카페/술로 분류.
- **M4 — 후기 기능:** `reviews-section.tsx` 추가, `localStorage`·소속 필터·삭제·평균 별점.
- **M5 — 수정 라운드(A1~C4):** 유형 필터 분리(A2), 다크 모드 단일화(B1), 점수 규칙 정리(B2), 규칙 기반 안내 문구(B3), README/메타데이터/eslint·tsconfig 정리(C계열).
- **M6 — 검증·배포:** `npm run lint / npm run build` 통과 확인, main push, Vercel 배포.

배포 확인 (2026-06-14)

- Vercel production을 커밋 `d73cbe6d1a0e0108e48207490d9a9c14858e77b8` 기준으로 재배포했다.
- 배포 상태는 `READY / PROMOTED` , 고정 URL은 <https://testlanding-theta.vercel.app> 이다.
- 공개 URL은 HTTP 200, 리다이렉트 0회로 응답했고 정적 HTML에서 "이 조건이 맞아서 추천", "이 장소 후기 쓰기", "후기", "KENTECH 전체 구성원"을 확인했다.

10. 현재 한계 (수정 후 기준)

A2 수정 이후 "선택 유형과 다른 유형이 결과에 섞인다"는 과거 문제는 **더 이상 발생하지 않는다**(시뮬레이션으로 확인). 대신 지금 시점의 실제 한계는 다음과 같다.

- **추천은 정적·결정론적이다.** 같은 입력엔 항상 같은 3곳. 시간대·요일·실시간 영업 여부를 전혀 모른다.
- **유형 분리의 이면.** 유형을 엄격히 거르기 때문에, 밥을 골랐을 때 맥락상 좋은 카페가 있어도 절대 후보로 올라오지 않는다(의도된 트레이드오프).
- **동률이 많다.** 밥·술 경로는 카페처럼 태그 가산점이 없어서 여러 곳이 같은 점수(예: 밥·저예산·가까움·빠름·혼밥 조합에서 14점 동률 다수)로 묶인다. 이때 순서는 JSON 배열 원래 순서(안정 정렬)로 갈려, 사실상 임의의 tie-break다.
- **거리는 진짜 거리 계산이 아니다.** 사용자가 고르는 범주값일 뿐이고 `walk` 필드도 "차량 이동권" 같은 서술형 설명이다.
- **후기는 공유되지 않는다.** `localStorage` 라 다른 사람·다른 기기에서는 보이지 않고, 브라우저 데이터를 지우면 사라진다.
- **데이터는 수기 큐레이션이다.** 공개 정보(DiningCode 등) 기반이라 최신 영업 상태와 어긋날 수 있다.

[확인 필요] (코드로 확정 불가)

- 사용한 AI 도구의 정확한 구성·역할 분담 (Codex / EasyNext / Cursor / Vercel CLI 등 — 흔적은 보이거나 누가 무엇을 했는지는 코드로 단정 불가)
- M1~M6 각 단계의 실제 날짜